event-oriented system using the adapted movies example, instead of making calls to API endpoints, messages are published to topics, which are received by components that have subscribed to those topics. The Pact framework is built around HTTP requests, which is a limitation of the tool when implementing message-based communication. We'll focus on a scenario that updates movie data via an event broker called Kafka. Kafka is an open source distributed messaging system. Each part of the contract testing framework will sit alongside different parts of the events infrastructure, so it's important to understand the events implementation. For example, a web client publishes a message to add a new movie. The mobile client sends an asynchronous message to the "movies" Kafka topic. A backend service subscribes to the "movies" topic and adds the new movie data to the database. The next time a user searches for movies, they will be able to see the new movie data in the search. To explain it another way, with all the different components shown in figure 8.2, the producer publishes an event to trigger the addition of a movie on the Kafka message broker. The message broker job is to pass data between data sources. In this example, the message broker attaches it to a specific topic. Then, the service subscribed to the topic receives a message to create a movie. The subscriber will then process the message and perform the required steps to add the movie.